Senior Engineering Checklist: The "Senior+" Production Standard

This checklist is the actionable companion to the *Exhaustive Engineering Assessment Guide*. It breaks down every "Senior+" standard into granular, verifiable steps.

1. Filesystem & Documentation

- [ ] **Clear Folder Structure**:
    - /requirements: Original project requirements and specs.
    - /docs/architecture: High-level system design diagrams.
    - /docs/data-flow: Specific logic flows (Ingestion, Retrieval).
    - /src: Production implementation (structured by service/domain).
    - /tests: Comprehensive testing suite (API, Performance).
    - README.md: Clear installation and evaluation guide.
- [ ] **Stakeholder PPT**: A briefing deck summarizing EDA, Design, Decisions, and Evaluation.
- [ ] **Readme Consistency**: Does the code and structure actually follow the Readme?

2. Architecture & Design Integrity

- [ ] **Architecture vs. Data Flow Distinction**:
    - **Architecture**: Highlights software components, microservices, and caches.
    - **Data Flow**: Shows logical movement (e.g., retrieval sequence). *Do not confuse the two*.
- [ ] **Production Scale Deployment**:
    - [ ] Does the architecture discuss **API Gateways, Load Balancers, and Kubernetes (K8s)**?
    - [ ] Clear distinction between what is in POC vs. Production.
- [ ] **Observability & MLOps**: Architecture explicitly includes monitoring, logging, and ML lifecycle layers.
- [ ] **Design Decisions**:
    - [ ] ADRs included explaining Pros/Cons for major choices (e.g., DECISIONS.md).
    - [ ] Trade-offs explicitly highlighted (e.g., **Accuracy vs. Performance**).
    - [ ] **Decoupling**: Can you swap your Vector DB (e.g., FAISS to Milvus) without rewriting the core API?

3. Implementation & Code Quality

- [ ] **Production Grade Code**:
    - [ ] **Zero print() Statements**: 100% usage of structured logging (JSON preferred).
    - [ ] **Clean Code**: No hardcoded secrets, and no monolithic "God" files.
    - [ ] **Microservices Representation**: Is the microservice architecture reflected in code boundaries/packages?
    - [ ] **Connection Pooling**: Always used for DB and downstream services (zero per-request TCP overhead).
    - [ ] **Input Validation**: Are API inputs/outputs validated using a schema (e.g., Pydantic)?
- [ ] **Containerization**: Working Dockerfile and docker-compose.yml for a "one-command" startup.
- [ ] **Resource Management**:
    - [ ] Memory-efficient processing (streaming/generators).
    - [ ] **Cold Start Optimization**: Are models and indices loaded at startup, not on the first request?
- [ ] **Error Handling**: Does the system handle missing files, empty data, or API timeouts gracefully?

4. Testing & Validation (Accuracy)

- [ ] **API Testing**: Automated tests for both Loading (Ingestion) and Retrieving (Search).
- [ ] **Performance Measurement**: Monitoring latency (p99) and throughput for retrieval.
- [ ] **Accuracy Validation**:
    - [ ] Clearly defined methodology for validating retrieval accuracy (e.g., **LLM-as-Judge** or custom rubrics).
    - [ ] **Ground Truth Dataset**: Is the data used for self-evaluation provided and documented?
    - [ ] **Metrics Summary**: Does documentation include a summary of evaluation results (Latency & Accuracy)?
- [ ] **ML Resiliency**:
    - [ ] **Local Fallback**: Working code for a local model (Flan-T5/Transformers) if OpenAI times out.
    - [ ] **Graceful Degradation**: System returns fallback results (e.g., keyword-only) if vector indexing fails.

5. Verification: The Reviewer's Lens (SME Checklist)

Dimension	"No" Verdict	"Borderline" Verdict	"Yes" Verdict

Correctness	Bugs in core flow / Brittle logic.	Functional, but brittle.	Robust, handles edge cases.
Architecture	Monolithic / Sphaghetti code. No clear separation of concerns.	Layered but tightly coupled (e.g. logic in routers).	**Modular Microservices**. Clean separation of DB, AI, and API layers.
Design Decisions	Ad-hoc technology choice without reasoning.	Choosing "popular" techs but unable to explain trade-offs.	**ADRs included**. Clear justification of cost vs. latency vs. complexity.
Performance	High-latency LLM calls for every trivial task. No caching.	Basic in-memory caching. Single-threaded ingestion.	**Optimized Pipelines**. SLMs for latency, Parallel ingestion, distributed caching.
Testing	Manual testing only. No scripts.	Basic unit tests for utility functions only.	**Comprehensive Suite**. Unit, Integration, and Load tests (locust/custom).
Evaluation	None (Manual checks).	Script-based logs.	**Automated Rubrics**. LLM-as-Judge or IR metrics (NDCG/MAP).
Scalability	Global shared state / No pooling.	Vertical scaling only.	**Cloud-Ready**. Stateless, horizontally scalable, connection pooled.
Reliability	No error handling / No fallbacks.	Basic try/except.	**Resilient**. Retry logic, Local fallbacks (T5), and circuit breakers.
Maintainability	Hardcoded keys / Improper layout.	Clean but lacking documentation.	**Self-Documenting**. Clean code, logic grouping, and Dockerized setup.
Observation	print() statements.	Basic log output.	**Production-Ready**. Structured logs & health checks.

6. Final Benchmarking (SME 'Yes' Grade)

- [ ] **Core Flow**: System is robust and handles "sad path" edge cases (empty data, API timeouts).
- [ ] **Architecture**: Clear separation between DB, AI, and API layers in a modular/service-oriented layout.
- [ ] **Design Decisions**: ADRs provide clear justification for tech choices based on cost, latency, and complexity.

- [ ] **Performance**: Ingestion is parallelized, and high-frequency tasks use optimized SLM models.
- [ ] **Testing**: Includes a comprehensive suite of Unit, Integration, and Load tests.
- [ ] **Evaluation**: Quality is measured using automated rubrics (LLM-as-Judge) or IR metrics.
- [ ] **Scalability**: Application is stateless, horizontally scalable, and utilizes connection pooling.
- [ ] **Reliability**: Implements retry logic, local model fallbacks, and circuit breakers.
- [ ] **Maintainability**: Code is self-documenting, grouped logically, and fully Dockerized.
- [ ] **Observation**: Production-ready structured logs and functional health check endpoints.

From: Meena Krishnan <meena.k@prodapt.com>
Sent: Friday, February 13, 2026 20:39
To: Vijay Ramachandran <vijay.r@prodapt.com>
Subject: RE: Assessment Guidelines

@Vijay Ramachandran

Tried to have some kind of checklist – can you please help me to have exhaust checklist

1 Have clear folder structure

Requirement

Architecture & Design

Data Flow

Implementation /Code

Testing

Readme

PPT briefing all

2 Don't confuse data flow and architecture - Both are different - For e.g. ingestion and retrieval flow is

3 Architecture should high light different software components/microservice's /cache etc

4	in Architecture if there are trade offs like accuracy vs performance high light
4	Add production scale deployment architecture and high light what is covered in POC -Like API Gate Kubernetes etc..
5	Explain design decisions and pros and cons
6	Have production grade code - No printfs - if its microservices architecture represent that in the co
7	If possible containerize the code
8	Highlight how to do testing & how to validate the accuracy - API tests for loading/retrieving and the p same?
9	How did we do evaluation (like LLM as judge)
10	In Architecture explain about observability /monitoring and ML Ops

From: Meena Krishnan
Sent: 13 February 2026 11:19
To: Vijay Ramachandran <vijay.r@prodapt.com>
Subject: RE: Assessment Guidelines

@Vijay Ramachandran

Sorry – looking for something simpler 😊 I will explain

From: Vijay Ramachandran <vijay.r@prodapt.com>
Sent: 13 February 2026 10:39
To: Meena Krishnan <meena.k@prodapt.com>
Subject: Assessment Guidelines

Meena,

Pls find below the guidelines. **This is based on the Assessment feedback provided by OpenAI in comparison with our candidate deliverable.**

Universal Engineering Assessment Guide: The "Senior+" Standard

This guide provides a framework for clearing technical assessments for senior or lead-level roles. It focuses on the distinction between "working code" and "production-ready systems."

---

1. Requirement Mastery & Strategic Mapping

- **The "Shadow" Requirements: Identify requirements that are implied but not explicitly stated (e.g., if there's an ingestion pipeline, a deletion/cleanup pipeline is always an implicit production requirement).**
- **Traceability Matrix: Map every feature back to a specific requirement. If you add a feature not asked for, ensure it doesn't distract from core deliverables.**
- **Edge Case First: Design for the "sad path." How does the system handle empty inputs, malformed data, or downstream service outages?**

2. Architectural Integrity (The "Systems" View)

- **Modularity & Decoupling: Avoid the "Big Ball of Mud." Use Design Patterns (Factory, Strategy) to separate concerns.**
- **Contract-First Design: Define clear interfaces (APIs, Schemas) before implementation.**
- **Latency vs. Intelligence Trade-off: For high-frequency tasks (intent classification), consider replacing expensive LLMs with specialized, lightweight ML models (Sklearn/FastText) to achieve <100ms response times.**
- **Resiliency & Local Fallbacks: Design for failure. If a remote API (OpenAI) fails, the system should have a local fallback model (e.g., Flan-T5, Sentence-Transformers) to maintain basic service.**

3. Production-Grade Implementation Standards

- **Resource Management:**
  - **Always use connection pooling for DBs. Opening/closing connections for every request is a major performance bottleneck.**
  - **Memory Efficiency: Use generators/streaming for large data processing instead of loading everything into RAM.**
- **Stateless Scaling: Ensure thread safety if using shared state. Avoid monolithic in-memory caches that trap state in a single process—prefer external stores (Redis) or stateless designs for horizontal scaling.**
- **Configuration: Use environment variables or centralized services (Pydantic Settings). Never hardcode API keys or credentials.**
- **Robust Data Processing: Never use brittle logic (fixed string slicing/happy-path regex). Use structured schema validation (Pydantic/JSON Schema) to handle messy real-world data.**

4. Advanced Data & AI/ML Operations (MLOps)

- **The Evaluation Framework: This is the biggest differentiator. Include an automated measurement system:**
    - **Quality: LLM-as-Judge, Rubric-based scoring, or IR metrics (NDCG, MAP, MRR).**
    - **Ground Truth: Provide the dataset used for self-evaluation.**
- **Cold Start & Warmup: Handle resource-heavy initializations (model loading, index building) during startup logic, not on the first request.**

5. Operational Excellence (DevOps/SRE)

- **Containerization: A** Dockerfile **is mandatory. A** docker-compose.yml **that sets up the entire stack is highly preferred.**
- **Observability:**
    - **Logging: Structured logs (JSON) with proper levels. Avoid** print()**.**
    - **Monitoring: Health check endpoints (/health, /ready) that check downstream connectivity.**
- **Advanced UX: Use patterns like Server-Sent Events (SSE) for streaming long-running search or recommendation results to improve perceived performance.**

6. Documentation & Decision Logic (The "Why")

- **Architecture Decision Records (ADRs): Document *why* you chose Technology A over Technology B. Showing the trade-offs (e.g., "Complexity vs. Performance") shows senior judgment.**
- **System Layout: Use file/directory structure to tell a story. Group by domain/component, not just by type.**

Verifying Success: The Reviewer's Checklist

Dimension	"No" Verdict	"Borderline" Verdict	"Yes" Verdict
Correctness	**Bugs in core flow / Brittle logic.**	**Functional, but brittle.**	**Robust, handles edge cases.**
Architecture	**Monolithic / Sphaghetti code. No clear separation of concerns.**	**Layered but tightly coupled (e.g. logic in routers).**	**Modular Microservices. Clean separation of DB, AI, and API layers.**
Design Decisions	**Ad-hoc technology choice without reasoning.**	**Choosing "popular" techs but unable to explain trade-offs.**	**ADRs included. Clear justification of cost vs. latency vs. complexity.**
Performance	**High-latency LLM calls for every trivial task. No caching.**	**Basic in-memory caching. Single-threaded ingestion.**	**Optimized Pipelines. SLMs for latency, Parallel ingestion, distributed caching.**

Testing	**Manual testing only. No scripts.**	**Basic unit tests for utility functions only.**	**Comprehensive Suite. Unit, Integration, and Load tests (locust/custom).**
Evaluation	**None (Manual checks).**	**Script-based logs.**	**Automated Rubrics. LLM-as-Judge or IR metrics (NDCG/MAP).**
Scalability	**Global shared state / No pooling.**	**Vertical scaling only.**	**Cloud-Ready. Stateless, horizontally scalable, connection pooled.**
Reliability	**No error handling / No fallbacks.**	**Basic try/except.**	**Resilient. Retry logic, Local fallbacks (T5), and circuit breakers.**
Maintainability	**Hardcoded keys / Improper layout.**	**Clean but lacking documentation.**	**Self-Documenting. Clean code, logic grouping, and Dockerized setup.**
Observation	print() **statements.**	**Basic log output.**	**Production-Ready. Structured logs & health checks.**

Disclaimer

This email contains proprietary, confidential and privileged information of Prodapt Solutions Private Limited and its affiliates (“Prodapt” or “We”) and is intended for the exclusive use of the named recipient (“Recipient”). If you are not the intended Recipient and have received this email by mistake, please reply to this email and follow with its deletion, so that we can ensure such a mistake does not occur in the future. It is strictly prohibited to share any part of this email with any third party, without an express written confirmation by Prodapt. All e-mail, if permitted by applicable law, sent to and from Prodapt is subject to archiving and review by someone other than the Recipient. We do not accept any liability for any errors or omissions in the contents of this e- mail nor do we represent that this email and any attachments is without any errors and virus free. In the event this email contains Personal Data of the Recipient, we would like to inform that we process Personal Data in accordance with our Privacy Policy.No employee/contractor is authorized to conclude any binding contract on behalf of Prodapt without an express written confirmation by an executive officer of Prodapt. Please do not print this email unless it is necessary. Every unprinted email helps the environment. Thank you for your co-operation

www.prodapt.com